SciWeaver: Context-Driven Dependency-Aware Multi-Agent Orchestration for Trustworthy Research

Mahady Hasan Rayhan*
mhr6wb@missouri.edu
University of Missouri
Columbia, Missouri, USA

Vani Seth*
vs4ky@missouri.edu
University of Missouri
Columbia, Missouri, USA

Pari Hirenkumar Patel
ppf2d@missouri.edu
University of Missouri
Columbia, Missouri, USA

Ashish Pandey
apfd6@missouri.edu
University of Missouri
Columbia, Missouri, USA

Upasana Roy
u.roy@missouri.edu
University of Missouri
Columbia, Missouri, USA

Sanjit Subhash
ssyht@missouri.edu
University of Missouri
Columbia, Missouri, USA

Prasad Calyam
calyamp@missouri.edu
University of Missouri
Columbia, Missouri, USA

Abstract

Scientific research needs iterative analysis, experimentation, and reasoning, which is time-consuming and resource-intensive, reducing the pace of scientific breakthroughs. LLM-based research copilots with tooling agents have been shown to accelerate this process, especially for multi-step research workflows that require trustworthy responses drawn from distributed knowledge/instrument sources. However, there is a need to improve the reliability of agentic tools' reasoning towards the research queries, since AI agents are resource-intensive and often hallucinate with untrustworthy, non-explainable responses. In this paper, we present 'SciWeaver', a dependency-aware research copilot that treats inter-agentic tool context as a key orchestrating factor for improving trustworthy response generation to queries. SciWeaver constructs a context-driven dependency graph that determines agents' dependency on other agents' outputs before and during execution using our proposed Probing-of-Thoughts methodology. It assesses responses from each agentic tool adaptively, probing deeper with increased context only when needed or recreating the dependency graph if agents' responses do not align with the research query. We evaluate the proposed system against existing multi-agent reasoning & orchestration frameworks and baselines such as GPT and Gemini, using metrics such as 'LLM-as-a-Judge' accuracy and retrieval quality. Our experimental results demonstrate that dependency-aware context propagation during tool orchestration significantly improves the response accuracy (>19% higher correctness score against all baselines) and coherence for complex multi-step research queries.

CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; **Natural language processing**; **Agentic AI**; **Explainable AI**;

Keywords

Trustworthy and Explainable AI, Dependency-Aware Orchestration, Multi-Agent Systems, Agentic-AI

ACM Reference Format:
Mahady Hasan Rayhan, Vani Seth, Pari Hirenkumar Patel, Ashish Pandey, Upasana Roy, Sanjit Subhash, and Prasad Calyam. 2027. SciWeaver: Context-Driven Dependency-Aware Multi-Agent Orchestration for Trustworthy Research . In *Proceedings of the 28th International Conference on Distributed Computing and Networking (ICDCN '27), January 4–7, 2027, Surathkal, India.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

*These authors contributed equally to this work.

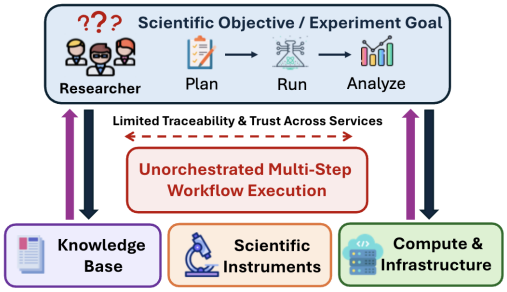


Figure 1: Unorchestrated multi-step interactions across agents, limiting workflow traceability and trustworthiness.

1 Introduction

The growing scale and complexity of modern scientific research have made it increasingly difficult for researchers to efficiently discover, synthesize, and act on relevant information. Modern scientific discovery operates across distributed ecosystems, where scientists access heterogeneous data, scientific instruments, and edge–cloud resources across institutions. In domains such as bioinformatics, materials science, biological science, and chemistry, researchers must integrate these distributed resources to achieve scientific discovery goals [1].

To address these challenges, recent efforts have explored LLM-based chatbot interfaces as research copilots, enabling scientists to interact with distributed scientific resources through natural language [2]. Multi-agent systems can support this complexity by assigning specialized agents for different scientific tasks. These interfaces are extended through the agentic AI paradigm, where specialized agents support tasks such as knowledge retrieval, scientific analysis, and resource selection while connecting literature services, laboratory instruments, and cloud–edge resources toward common scientific goals [3]. Complex research queries can be decomposed into multi-step workflows spanning heterogeneous data, tools, and computing resources. Frameworks such as ReAct [4] and Reflexion [5] further enable agents to reason, act, and adapt during execution, increasing their autonomy within these workflows.

However, as shown in **Figure 1**, without effective orchestration, multi-step agent interactions become difficult to track and verify, limiting workflow traceability and trustworthiness. Individual

agents may produce incomplete evidence, incorrect assumption, or reasoning errors that propagate through inter-agent dependencies and affect downstream decisions leading to misinformation, privacy leakage, adversarial threats, and misconfigured resource [6]. This fragmented interaction threatens scientific integrity by limiting workflow traceability and reliability. Thus, an open challenge is enabling trustworthy multi-agent execution that preserves inter-agent dependencies and context while adapting to failures, beyond existing task coordination and message passing approaches. Accordingly, our threat model considers failures arising from agent interactions and distributed resources.

In this paper, we address the above limitations and present *SciWeaver*, a researcher-centric context-driven multi-agent orchestration framework for scientific discovery. Our key hypothesis is that managing optimal/adaptive context across agents can improve their trustworthiness and, as a results improve integrity of scientific workflows. *SciWeaver* consists of two primary layers: First, an **Orchestrator:** Responsible for preparing an execution plan and has two primary components: a *User Profiler*, which is responsible for maintaining a dynamic user-intent profile for response generation that adapts across queries, and *Query Decomposer*, which is responsible for query understanding, task decomposition, dependency planning, and orchestration across specialized agents. Second, **PoT (Probing-of-Thought):** a dynamic and failure-aware orchestration mechanism built for adaptive context-passing and DAG update during execution.

We evaluate *SciWeaver* in a bioinformatics based scientific discovery environment, which includes four specialized agents for different research tasks: a Knowledge Discovery Agent, a Scholar Recommender Agent, a Jupyter Notebook Agent, and a Cloud Solution Agent. Collectively, these agents provide capabilities for retrieval, recommendation, computational resource selection, and assistance. We compare *SciWeaver* against four baselines consisting of two single-model LLMs (GPT and Gemini) that process queries in a single pass without decomposition, and two linear multi-agent pipelines that execute tasks sequentially 'with' and 'without' upstream context propagation. We also study the impact of probing with adaptive context and dynamic DAG change towards enhancing response quality measured using metrics such as correctness, specificity, and coherence.

The rest of this paper is organized as follows. Section 2 reviews related work. Section 3 formalizes the key system components. Section 4 describes the methodology for the orchestration and our PoT approach. Finally, Section 5 presents our experimental setup and results. Section 6 concludes the work.

2 Related Work

2.1 Trustworthy Decision-Making Frameworks

Structured reasoning frameworks have become increasingly prominent for supporting decision-making in LLM-based agents, strengthening both problem-solving and agent reliability. Methods such as Chain-of-Thought (CoT) [7] and multi-hop dependency graphs [8] make LLM reasoning more explicit and controllable, improving the transparency of the decisions that follow. Building on these reasoning foundations, [9] expanded reflective reasoning into the multi-agent tool-learning setting by introducing intra-reflection; they evaluate intended actions before execution to reduce error propagation, and inter-reflection, which in turn enables trajectory correction using observations from other agents. Collectively, these approaches represent reasoning as structured, verifiable computational processes, with recent work introducing adaptive inference to scale or truncate computation based on task complexity, reducing cost while preserving quality [10]. Our Probing-of-Thoughts (PoT) methodology extends this approach by enabling adaptive reasoning at the granularity of individual agentic tool calls, with controlled context passing and defined failure modes. This supports trustworthy multi-agent execution by making inter-agent dependencies explicit and preventing intermediate errors from propagating to downstream reasoning.

2.2 Multi-Agent Orchestration

Authors in [11] propose teaching models to determine when and how to activate tools without task-specific examples. Authors in [12] extended this idea to multi-model orchestration, employing an LLM as a controller to organize tasks and choose models. Similarly, [13] showed that LLM agents with observation, planning, and reflection can exhibit long-horizon behavior in multi-agent settings. However, existing approaches largely rely on message passing without modeling how intermediate reasoning constrains downstream decisions, particularly in scientific workflows involving distributed instruments, data sources, and edge–cloud resources. Although adaptive orchestration improves agent sequencing [14], it primarily focuses on task scheduling rather than dependency and context propagation across agents. In contrast, *SciWeaver* explicitly represents inter-agent dependencies through a DAG and propagates relevant context across dependent reasoning stages. Combined with PoT, the orchestration can probe intermediate outputs and adapt the execution path when failures or information gaps arise.

3 System Components

SciWeaver is organized around two components: (i) an Orchestrator that decides which agents to invoke for a given query, routes and supervises their execution, and (ii) specialized AI agents that interact with distributed knowledge sources. We use an exemplar bioinformatics workflow to demonstrate *SciWeaver*, where scientific tasks span distributed data sources and edge–cloud computing resources for data analysis and computational execution.

3.1 Orchestrator

The Orchestrator processes queries into *SciWeaver*, which uses Retrieval-Augmented Generation (RAG) powered by LLMs. It receives the user's natural-language query, decides which specialist agents are required, makes their data and reasoning dependencies explicit through an execution plan, and, in cooperation with the Probing-of-Thought layer, tracks intermediate outputs to support reliable execution. The Orchestrator itself is composed of two cooperating sub-components 1) a *User Preference Profiler* that infers *how* the user prefers to receive information and 2) a *Query Decomposer* that translates *what* is being asked into sub-queries.

User Preference Profiler: The User Preference Profiler maintains a dynamic, lightweight representation of the user's research style across queries, so that user context is preserved across downstream decomposition and synthesis. This follows the same principle as learner-state modeling in intelligent tutoring, where a knowledge

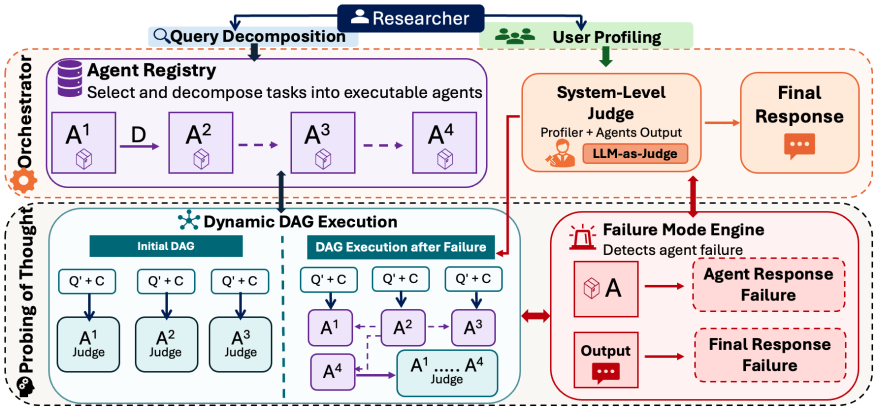


Figure 2: SciWeaver architecture: (i) the Orchestrator layer, which builds multi-agent workflows, and (ii) the Probing-of-Thoughts (PoT) layer which dynamically executes and adapts the DAG.

graph of each learner's cognitive state is maintained to adapt guidance across interactions [15]. For each incoming query, the profiler emits a probability distribution over three intent dimensions derived from Bloom's Revised Taxonomy [16] i.e., *Procedural* - the user wants code, pipelines, or step-by-step instructions, *Analytical* - the user wants comparisons, trade-offs, or literature evaluation, and *Conceptual* - the user wants definitions, mechanisms, or theoretical explanations. We refer to this triplet as the *PAC* vector. The per-query estimate is accumulated into a persistent session profile that drifts toward the user's recent behavior. This profile is later consumed at two downstream points: (1) it conditions the Query Decomposer to guide decomposition toward the user's preferred mode; and (2) it weights the final answer-synthesis step to control how strongly each agent's contribution surfaces in the response.

Query Decomposer: transforms a complex, possibly multi-faceted user query into a traceable, dependency-aware execution plan for multi-agent execution. It proceeds in three steps. *First*, an LLM extracts salient keywords and phrases from the input query and resolves pronouns and demonstratives into explicit referents, so that every downstream sub-task is self-contained. *Second*, the extracted phrases are matched against the metadata describing each available agent, partitioning the query into focused sub-queries aligned with the most suitable agent. *Third*, inter-sub-query dependencies are made explicit, and the sub-queries plus their dependency edges are organized into a directed acyclic graph (DAG) execution plan.

3.2 Specialized AI Agents

SciWeaver integrates four specialized agents derived from the OnTimeRecommend science gateway system [1].

Knowledge Discovery Agent: The Knowledge Discovery Agent surfaces domain-specific scientific knowledge by chaining two internal recommendation phases. The *first* phase identifies and characterizes the domain and subdomain of the query, The *second* phase performs targeted retrieval within that domain, returning papers, software tools, and datasets. The Knowledge discovery outputs, such as topics and author names, help the Scholar Recommender Agent identify relevant researchers, while identified tools and datasets guide the Notebook Agent.

Scholar Recommender Agent: The Scholar Recommender Agent identifies active researchers whose expertise matches the user's target domain. It queries the Semantic Scholar API to retrieve researcher profiles and ranks them by domain relevance, publication recency, and citation impact. The agent is designed to receive enriched input from the Knowledge Discovery Agent such as topic domains and author names which is then injected into the Scholar Recommender Agent's search parameters, enabling subfield-level collaborator matching. Its output is a ranked list of researcher profiles including affiliations, citations, and recent publication highlights.

Cloud Solution Agent: The Cloud Solution Agent is designed to recommend cloud compute-resource configurations drawn from public and community cloud platforms. It leverages prior knowledge of the compute resources and tools required to execute domain-specific analytics. The agent defaults to pipeline type and compute estimate as structured input provided by the Notebook Agent and matches them against a catalog of available instance configurations. Each recommendation is deployable and includes the instance details such as CPU count, memory, and estimated cost.

Notebook Agent: The Notebook Agent retrieves and transforms relevant Jupyter notebooks from the science gateway's notebook repository to meet the user's computational-workflow needs. The agent extracts structured outputs from each matched notebook so that downstream agents can consume them programmatically rather than parsing free-form text. Each Notebook Agent response contains the workflow type, compute requirements, and relevant notebooks.

4 Methodology

SciWeaver as illustrated in **Figure 2** comprises two cooperating layers: **Orchestrator layer**: performs context-aware orchestration through query decomposition, user profiling, and dependency-aware execution-plan construction; and **Probing-of-Thought (PoT) layer**: dynamically executes and adapts the DAG-based reasoning process while monitoring intermediate agent outputs and the final response through a system-level LLM-as-a-Judge.

Trustworthiness in *SciWeaver* is ensured as a property that both layers maintain at every boundary of the execution graph. We use the term *trustworthy orchestration* to denote three properties that the architecture enforces jointly: i) *Verifiability*: every agent output is scored against the specification of its downstream consumer before it is allowed to propagate; ii) *Traceability*: the context that crosses each DAG edge is produced by a deterministic, inspectable

atomization step; and iii) *Calibrated abstention*: sub-tasks that cannot be repaired within a bounded retry budget are surfaced to the researcher as low-confidence.

4.1 Agent Orchestrator Layer

The Orchestrator transforms a natural-language researcher query into a typed, dependency-aware execution plan. It consists of three sequentially coupled components: a *User Profiler*, a *Query Decomposer*, and a *DAG Constructor*.

4.1.1 User Profiling: SciWeaver maintains a dynamic user-intent profile that adapts across queries and conditions three stages of the pipeline which are 'query decomposition', 'agent execution behavior', and 'final-response synthesis'.

Intent space: Each incoming query is classified into a three-dimensional intent space derived from Bloom's Revised Taxonomy [16]. These dimensions capture the primary modes of interaction discussed in Section 3.1.

Classifier: The intent classifier is implemented as a Multinomial Naive Bayes (MNB) model trained on a manually curated corpus of 32 bioinformatics queries (12 Conceptual, 10 Procedural, 10 Analytical) drawn from five predefined bioinformatics subdomains.

The MNB model is fit with Laplace smoothing ($\alpha = 1.0$), trained once at system startup, and then subsequent runs load the saved artifact without retraining. At inference, the model returns a soft probability distribution over all three classes simultaneously, so that downstream weighting reflects mixed-intent queries. For each query, the classifier produces a probability vector representing the estimated PAC distribution for that query. *SciWeaver* maintains a persistent profile state $\mathbf{s} \in \mathbb{R}^3$, initialized to a uniform distribution and updated after each query using an Exponential Moving Average (EMA): $s_d \leftarrow (1-\lambda)s_d + \lambda\hat{p}_d$, where $d \in \{P, A, C\}$ and $\lambda = 0.4$ controls the influence of recent interactions. After each update, $\mathbf{s}$ is renormalized to sum to 1. This formulation lets the profiler adapt to evolving user behavior without retaining the full conversational history.

Conditioning the pipeline: The PAC profile conditions the response generation pipeline at two points: (1) *Decomposition-time hint:* The dominant intent dimension of the current query is converted into a compact natural-language routing hint injected into the Decomposer's prompt. This biases the task graph toward the inferred user intent (procedural/analytical); and (2) *Synthesis-time weights:* The accumulated profile state s is propagated into the system-level synthesis step, where it acts as the weight vector that the Orchestrator LLM uses to combine individual agent outputs. The system generates a specific style instruction tailored to whichever profile(procedural/analytical) is dominant.

4.1.2 Query Decomposition: The Decomposer converts the natural-language query into a set of structured, typed sub-goals suitable for multi-agent execution. The query is passed to an LLM-based decomposition module conditioned on the user's PAC routing hint. The decomposition prompt enforces three constraints: (1) pronouns and demonstratives are resolved into explicit referents so that each sub-goal is self-contained; (2) each sub-goal contains exactly one intent category to ensure unambiguous downstream routing; and (3) inter-goal dependencies are represented explicitly through placeholder references of the form `[result of goal M]` together with a corresponding `[depends_on]` field. The first matching rule routes the sub-goal to one of four agents. Required agent-specific metadata is attached during routing. Cross-sub-goal dependencies are extracted at this stage, producing a dependency-aware task list for DAG construction.

Context routing via CUGCR: When one agent's output must be passed to another, the raw text is often too long or off-topic for the downstream consumer. *SciWeaver* applies a context compression module, *Causal Utility-Gated Context Routing (CUGCR)* that filters the upstream output down to only the information atoms the downstream agent actually requires, subject to a per-agent token budget. CUGCR runs a multi-stage compression pipeline: (1) A *token-budget gate* forwards the raw context unchanged when it already fits the downstream agent's budget; and (2) A regex parser deterministically splits the upstream output into typed units - `CODE_BLOCK`, `NAMED_ENTITY`, and `PROSE_CLAIM`. These units are assessed against the next specialized agent tasks to include or partially drop from being passed.

4.1.3 DAG Construction: The Orchestrator compiles the dependency-aware sub-goal list into a typed execution DAG $G = (V, E)$, where nodes $v_i \in V$ are sub-tasks and directed edges $(v_i, v_j) \in E$ encode data dependencies. For every agent selected by the Decomposer, the Orchestrator issues a single LLM inference call. Each call receives the agent's tool description and assigned sub-task, and produces a typed node specification capturing: (1) what the agent *produces* for exmaple output fields and types; (2) what the agent *requires* as input for fulfilling its function; (3) a *quality bar* that the output must meet to be useful downstream. The quality-bar allows the System-Level Judge (Section 4.2.1) to assess downstream fitness over abstract correctness. The resulting nodes and edges form the initial DAG, executing independent nodes immediately while others wait for CUGCR-compressed inputs before passing the plan to the Probing-of-Thought layer.

4.2 Probing-of-Thought (PoT) Layer

The PoT layer executes the DAG, monitors outputs and synthesizes the final response, and replans upon failure using a central *System-Level Judge* (**Figure 2**), a *Trust Diagnosis Engine* for failure attribution, and a *Re-Planner* to revise inputs or DAGs for execution.

4.2.1 Trust Verification via System-Level Judge: While agents perform lightweight internal sanity checks, *SciWeaver* relies on a single LLM-based System-Level Judge for all orchestration assessments, which acts as the system's verification boundary, i.e., an output that does not clear the Judge cannot cross a DAG edge or reach synthesis. The Judge operates at two points: **(1) Per-agent assessment** For each node v_i, the Judge evaluates the sub-task s_i, output r_i, and downstream role specification $Q(i \to j)$, emitting a fitness score $\hat{s}_i = (0.35c + 0.30d + 0.25a + 0.10l)/10 \in [0, 1]$, where c, d, a, and l are 1 to 10 ratings for completeness, depth, accuracy, and clarity. If $\hat{s}_i < \tau$ (we use $\tau = 0.7$), it provides a structured failure descriptor. It explicitly assesses downstream fitness rather than isolated correctness; a technically correct output may still fail if it lacks the specificity required by its downstream consumer; and **(2) Final-response assessment** After the Orchestrator LLM synthesizes a unified response, the Judge scores how well its style and emphasis match the user's PAC profile $\mathbf{s}$.

4.2.2 Trust Failure Attribution: Failures detected by the System-Level Judge fall into two top-level modes: **(1) Wrong output at**

agent-level: one or more agents produced an output that does not satisfy the quality bar of its sub-task. The affected agents must be re-executed with enriched context, and **(2) Weight mismatch at system-level**: the collective response is technically correct, but the composition does not reflect the user's PAC profile; for instance, a strongly procedural user receives a primarily conceptual synthesis.

For agent-level (*wrong-output*) failures, the Trust Diagnosis Engine assigns one of four sub-types to refine the re-execution context: 1) *Specificity failure*: output is too generic for the downstream agent; 2) *Coverage failure*: required fields are missing; 3) *Relevance failure*: output drifted from the objective; 4) *Knowledge boundary*: the agent cannot improve the output via reformulation, marking the sub-task as unrecoverable. These sub-types are not derived from the Table 2 metrics; the Judge assigns exactly one, with a one-sentence rationale, in the same call that produces $\hat{s}_i$ (e.g., low d signals a specificity failure and low c a coverage failure).

4.2.3 Adaptive DAG Replanning: When a failure is detected, *SciWeaver* does not restart the DAG from scratch since doing so would discard the useful partial information already produced by the upstream agents. Instead, the PoT layer chooses one of two adaptive responses, conditioned on which evaluation point led to failure: **(1) Agent-level failure:** The failing node is preserved, but the context C passed to it is enriched, i.e, the Re-Planner expands C to include additional units from upstream outputs by relaxing the CUGCR token budget for this node and reformulates the sub-query Q' according to the failure sub-type on agent level. Specifically, *specificity* failures trigger narrower, more entity-targeted reformulations; *coverage* failures trigger broader reformulations that explicitly list missing fields; and *relevance* failures trigger reanchored reformulations that restate the sub-task objective; and **(2) Final-response failure:** The Re-Planner modifies the DAG itself based on the PAC weight vector $\mathbf{s}$, either by inserting additional dependency edges between agents so that, e.g., the Cloud Solution Agent now receives Knowledge Discovery output it previously did not consume or by adjusting the CUGCR compression budgets to widen the context channels between specific node pairs. The PAC weights are then re-applied so that the synthesis step recombines the (re-)executed outputs with the user's profiled intent emphasis.

5 Evaluation

Our evaluation setting reflects a representative bioinformatics workflow that spans distributed data sources, literature repositories, and edge-cloud computing resources. Herein, we evaluate *SciWeaver*'s component- and system-level performance across four stages. *First*, we independently evaluate the four specialized AI agents. *Second*, we assess the User Profiler's intent prediction. *Third*, we evaluate the end-to-end system on complex bioinformatics workflows. *Finally*, we analyze the impact of orchestration, adaptive refinement, and dynamic DAG expansion under PoT.

Metrics: Outputs are evaluated using domain-tailored metrics (e.g., reasoning quality, retrieval grounding, code correctness, orchestration accuracy, and infrastructure validity). Offline scores are normalized to a 0–10 scale and aggregated into a composite GS-Score for system-level comparison. We also report pass rate, latency, keyword-hit accuracy, and grounding statistics where applicable. To mitigate evaluation bias, online and offline judges use different model families. The profiler's MNB intent classifier achieves 68.8% leave-one-out accuracy (macro-F1 0.66) vs. a 37.5% majority-class baseline; its output serves only as a soft PAC prior, further smoothed by the EMA profile.

Table 1: Performance of specialized execution agents.

Agent	Pass Rate	Score (0-10)	Keyword Hit	Latency (s)
J-Code	100%	9.20	95%	3.0
I-Cloud	90%	8.70	66%	5.7
K-Rag	90%	7.70	82%	4.8
I-Scholar	60%	6.20	N/A	3.4

5.1 Agent-Wise Evaluation

We evaluate the four specialized agents independently to establish baseline performance prior to orchestration. These include: (i) K-Rag for literature synthesis, (ii) I-Scholar for researcher discovery, (iii) J-Code for Jupyter scripts, and (iv) I-Cloud for infrastructure. An LLM-based judge assesses outputs by assigning a raw score between 1 and 10, which is then normalized to a Node Quality Score (NQS) on a 0–1 scale. A query is considered successful if NQS > 0.75. We also report pass rate, keyword-hit accuracy, and end-to-end latency. As shown in **Table 1**, most agents show strong component-level performance. The Scholar agent achieves lower scores because its current reliance on NSF-supported metadata struggles to capture the broader, more ambiguous scholarly landscape.

5.2 Profiler Ablation Study

While previous experiments evaluate agents independently, the overall quality of *SciWeaver* also relies on orchestration conditioning. To evaluate this, we ablate the user profiler on a 30-query benchmark while keeping all other components constant. **Table 2** shows that enabling the profiler improves the weighted GS-Score from 3.88 to 4.25 (+0.371, +9.6%; 95% CI [+0.161, +0.583], $p < 0.01$, Wilcoxon signed-rank test). This improvement is consistent, outperforming the ablated system in 22 of 30 queries (7 losses, 1 tie; binomial test $p = 0.016$).

Table 2: Profiler ablation on the 30-query benchmark.

Condition	N	Mean (eq.)	GS-Score (wt.)	Std	p vs. Ours
SciWeaver + Profiler	30	**4.60**	**4.25**	0.86	—
SciWeaver (no profiler)	30	4.34	3.88	0.78	0.003 **

LLM-as-a-Judge evaluations (**Figure 3**) further highlight these gains. Enabling the profiler yields the largest improvements in Grounding (+0.50) and Specificity (+0.43), followed by Correctness (+0.27) and Code Quality (+0.27). The changes to Coherence (+0.16) and Completeness (−0.07) are minimal. Overall, incorporating user-specific context significantly improves contextual alignment and relevance.

5.3 System-Level Evaluation

We evaluate *SciWeaver* on exemplar multi-step bioinformatics workflows to determine if context-driven multi-agent coordination and Probing of Thought (PoT) refinement improve upon existing baselines. We compare against (i) LLM-only systems (GPT-5.4, Gemini Flash Latest) and (ii) ReAct pipelines (with/without context), using the same 30-query benchmark and evaluation rubric.

Table 3: System-level comparison of SciWeaver against baselines and ablations via LLM-as-a-judge (scores 0–10).

Model	Global Score (GTS)	Correctness	Completeness	Grounding	Specificity	Code Quality	Coherence
GPT-5 Baseline	4.43	3.77	3.57	1.27	4.30	1.80	6.27
Gemini-3 Baseline	3.88	4.37	3.73	1.80	5.10	2.80	6.80
ReAct (No Context)	6.05	4.63	6.00	5.97	6.87	3.47	7.97
ReAct (With Context)	6.31	4.83	6.33	6.27	7.10	3.77	8.10
SciWeaver (Ours - with PoT)	**6.77**	**5.77**	**6.57**	**6.57**	**7.53**	**4.77**	**8.40**
SciWeaver (Ours - without PoT)	**6.31**	**5.20**	**6.00**	**6.03**	**7.20**	**4.70**	**8.00**



Figure 3: Profiler ablation study: (a) mean score changes, and (b) score differences after enabling the profiler.

As shown in **Table 3**, *SciWeaver* achieves the strongest overall performance, improving the global score (GTS) from 6.31 (ReAct with context) to 6.77. Notable gains in Code Quality (+1.0), Grounding (+0.30), and Specificity (+0.43) demonstrate that specialized agents, retrieval grounding, and adaptive orchestration yield more accurate responses. This highlights the limitations of existing approaches: LLM-only baselines suffer from single-pass generation, while ReAct systems underperform despite iterative reasoning. These results suggest that reasoning alone is insufficient without explicit task decomposition, specialized execution roles, and adaptive context passing. **PoT Ablation Study:** While previous experiments evaluate the overall framework, this analysis focuses on the Probing-of-Thought (PoT) mechanism, which extends the static execution graph. We ablate PoT on the same 30-query benchmark while keeping all other components unchanged. As shown in **Table 3**, enabling PoT improves the Global Score (GTS) from 6.31 to 6.77. The largest gains occur in Correctness (+0.57) and Completeness (+0.57), indicating that iterative refinement recovers missing information and corrects reasoning errors. Grounding and Specificity also improve, as dynamic branching explores additional evidence paths. Code Quality sees only a modest gain (+0.07) since the standalone agent already performs well, while Coherence remains consistently high.

Overall, PoT provides meaningful benefits beyond static orchestration; while the orchestrator routes tasks, PoT effectively validates, refines, and expands intermediate reasoning. Thus, by catching and correcting these failures before they cross a DAG edge, PoT directly limits the propagation of incorrect or incomplete intermediate outputs, preserving the traceability and integrity of the resulting scientific workflow.

6 Conclusion

We presented *SciWeaver*, a multi-agent orchestration framework for trustworthy query resolution across distributed resources. Its *Orchestrator* combines a User Profiler, an LLM-driven Query Decomposer that builds a typed dependency-aware DAG, and CUGCR, which routes only the information each downstream agent requires. Its *Probing-of-Thought* layer uses a System-Level Judge to verify intermediate and final outputs, and a Re-Planner that either enriches a failing agent's context or replans the DAG. In a bioinformatics setting, SciWeaver outperforms single-model and linear multi-agent baselines on the Global Score, with notable gains in correctness, grounding, and specificity. Future work includes improving cross-domain generalization through adaptive intent profiling and continual learning, expanding the agent registry (e.g., statistical analysis, visualization, database querying), and integrating curated tool documentation, live literature, and scientific databases to further improve grounding and specificity.

GenAI Disclosure: AI-assisted tools were used in limited capacity for template code, baseline workflows, and debugging.

References

[1] Komal Vekaria, Prasad Calyam, Sai Swathi Sivarathri, Songjie Wang, Yuanxun Zhang, Ashish Pandey, Cong Chen, Dong Xu, Trupti Joshi, and Satish Nair. Recommender-as-a-service with chatbot guided domain-science knowledge discovery in a science gateway. *Concurrency and Computation: Practice and Experience*, 33(19):e6080, 2021.

[2] Upasana Roy, Vani Seth, Mahady Hasan Rayhan, Ashish Pandey, Mauro Lemus Alarcon, Matthew Maschmann, and Prasad Calyam. Closed-loop cnt growth using retrieval augmented generation and human feedback. In *2025 IEEE 5th International Conference on Human-Machine Systems (ICHMS)*, pages 128–133. IEEE, 2025.

[3] Yuheng Cheng, Ceyao Zhang, Zhengwen Zhang, Xiangrui Meng, Sirui Hong, Wenhao Li, Zihao Wang, Zekai Wang, Feng Yin, Junhua Zhao, et al. Exploring large language model based intelligent agents: Definitions, methods, and prospects. *arXiv preprint arXiv:2401.03428*, 2024.

[4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[5] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.

[6] Vani Seth, Srikar Alla, Sd Md Abidar Rahman, Ali Shiri Sichani, and Prasad Calyam. Altr: Adaptive defense framework against adversarial llm threats in critical-domain applications. In *2026 IEEE International Conference on Human-Machine Systems (ICHMS)*, pages 602–609. IEEE, 2026.

[7] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.

[8] Roland Oruche, Alphaeus Dmonte, Vani Seth, Zian Zeng, Yuanxun Zhang, Marcos Zampieri, and Prasad Calyam. Disentangling complex questions in llms via multi-hop dependency graphs. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, pages 2243–2252, 2025.

[9] Zikang Guo, Benfeng Xu, Xiaorui Wang, and Zhendong Mao. Mirror: Multi-agent intra-and inter-reflection for optimized reasoning in tool learning. *arXiv preprint arXiv:2505.20670*, 2025.

[10] Ruofan Zhang, Bin Xia, Zhen Cheng, Cairen Jian, Minglun Yang, Ngai Wong, and Yuan Cheng. Dart: Difficulty-adaptive reasoning truncation for efficient large language models. *arXiv preprint arXiv:2511.01170*, 2025.

[11] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

[12] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Advances in Neural Information Processing Systems*, 36:38154–38180, 2023.

[13] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of

human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.

[14] Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, et al. Multi-agent collaboration via evolving orchestration. *Advances in neural information processing systems*, 38: 165025–165059, 2026.

[15] Mahady H. Rayhan, Gabriel R. Edwards, Ashish Pandey, Vani Seth, Nilesh Salvi, and Noah Glaser. Sage: Student-focused adaptive guidance engine for intelligent tutoring with regulated learning. In *2026 IEEE International Conference on Human-Machine Systems (ICHMS)*, Singapore, Singapore, 2026. IEEE. doi: 10.1109/ICHMS69701.2026.11602253.

[16] Lorin W. Anderson and David R. Krathwohl, editors. *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. Longman, New York, 2001. ISBN 9780801319037. URL https://www.jstor.org/stable/42926529.